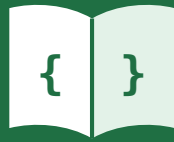

Modül 01 — Spring Core Container: IoC ve Bağımlılık Enjeksiyonu

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Inversion of Control (IoC)	2
2.2 Bağımlılık Enjeksiyonu (DI)	2
2.3 Bean'ler Nasıl Bulunur?	3
2.4 Container'ın Başlangıç Akışı	3
3. Adım Adım Örnekler	3
3.1 Constructor ve Setter Injection	4
3.2 @Component, @Bean ve @Configuration Proxy Modu	5
3.3 Bean Scope'ları	6
3.4 Yaşam Döngüsü Callback'leri	8
3.5 Profiller ve Koşullu Bean'ler	9
3.6 BeanRegistrar ile Programatik Kayıt (Spring Framework 7)	11
3.7 Uygulama Olayları (Events)	12
3.8 AOP ile Kesişen İlgiler	13
4. Sık Yapılan Hatalar ve En İyi Pratikler	14
5. Özet	15
6. İleri Okuma	15

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Inversion of Control (IoC) ve Dependency Injection (DI) kavramlarını açıklamak
- Constructor ve setter injection arasında doğru seçimi yapmak, field injection'dan kaçınmak
- @Component ile @Bean arasındaki farkı ve @Configuration sınıflarının proxy davranışını anlamak
- Singleton ve prototype scope'larını, "singleton içinde prototype" tuzağını ve çözümünü bilmek
- Bean yaşam döngüsü callback'lerini (@PostConstruct, @PreDestroy, SmartLifecycle) kullanmak
- Profillerle ve @ConditionalOnProperty ile ortama göre bean seçmek
- Spring Framework 7 ile gelen BeanRegistrar ile bean'leri programatik olarak kaydetmek
- Uygulama olaylarını (event) yayınlamak ve dinlemek
- Kesişen ilgileri (cross-cutting concerns) bir aspect ile ayırmak

Ön koşullar: Modül 00 (kurulum, modern Java) · **Tahmini süre:** 4 saat

2. Kavramlar

2.1 Inversion of Control (IoC)

Klasik kodda bir nesne ihtiyaç duyduğu nesneleri kendisi oluşturur (`new BookCatalog()`). IoC'de bu kontrol tersine döner: nesneleri **Spring container** oluşturur, birbirine bağlar ve yaşam döngülerini yönetir. Sizin sınıfınız yalnızca neye ihtiyacı olduğunu söyler.

Container'ın yönettiği her nesneye **bean** denir. Spring Boot uygulamasında container, `SpringApplication.run(...)` çağrısıyla oluşan `ApplicationContext`'tir.

2.2 Bağımlılık Enjeksiyonu (DI)

DI, IoC'nin uygulanış biçimidir: bir bean'in bağımlılıkları dışarıdan verilir. Üç yol vardır:

Yöntem	Ne zaman?
Constructor injection	Varsayılan tercih. Zorunlu bağımlılıklar, final alanlar, Spring'siz test
Setter injection	İsteğe bağlı (opsiyonel) bağımlılıklar

Yöntem	Ne zaman?
Field injection (@Autowired alan üzerinde)	Kullanmayın. Test edilmesi zor, bağımlılıklar gizli, alanlar final olamaz

2.3 Bean'ler Nasıl Bulunur?

- **Component scanning:** @SpringBootApplication sınıfının paketi ve alt paketleri taranır. @Component, @Service, @Repository, @Controller ile işaretli sınıflar bean olur.
- **@Bean metotları:** @Configuration sınıfındaki metotların dönüş değerleri bean olur. Kaynak kodunu değiştiremediğiniz sınıflar (ör. java.time.Clock) için kullanılır.
- **Programatik kayıt:** BeanRegistrar ile, döngü ve koşul içeren sıradan Java koduyla (bölüm 3.6).

2.4 Container'ın Başlangıç Akışı

1. Bean tanımları toplanır (scanning, @Bean, registrar'lar); koşullar (@Profile, @Conditional...) değerlendirilir.
2. Singleton bean'ler oluşturulur, bağımlılıklar enjekte edilir.
3. @PostConstruct metotları çalışır; gerekiyorsa proxy'ler (AOP) oluşturulur.
4. SmartLifecycle bean'leri başlatılır, ApplicationRunner'lar çalışır, uygulama hazırdır.
5. Kapanışta SmartLifecycle durdurulur, @PreDestroy metotları çalışır.

i Not

Singleton, Spring'in varsayılan scope'udur: container her bean'den **bir tane** oluşturur ve herkes aynı nesneyi paylaşır. Bu yüzden singleton bean'ler durum (state) tutmamalı ya da thread-safe olmalıdır.

3. Adım Adım Örnekler

Modülün tüm örneklerini, ders sırasıyla çalıştıran bir tur (LessonTour) vardır:

```
./mvnw -pl modules/01-core-container/lesson spring-boot:run
```

Her bölüm, konsolda == 3.x başlığıyla kendi çıktısını basar. Tüm örnekler com.springbootedu.corecontainer.paketinin altındadır.

3.1 Constructor ve Setter Injection

Amaç: Zorunlu bağımlılıkları constructor ile, opsiyonel bağımlılıkları setter ile almak.

BookService iki zorunlu bağımlılığı constructor'dan alır. Tek constructor olduğu için @Autowired yazmaya gerek yoktur:

```
@Service
public class BookService {

    private final BookCatalog catalog;           // final: set once, never null
    private final ApplicationEventPublisher events;

    // A single constructor needs no @Autowired: Spring calls it and passes the beans.
    public BookService(BookCatalog catalog, ApplicationEventPublisher events) {
        this.catalog = catalog;
        this.events = events;
    }
}
```

ReportPrinter ise bir ReportFooter bean'i **varsa** onu kullanır. @Autowired(required = false) sayesinde bean yoksa setter hiç çağrılmaz:

```
@Component
public class ReportPrinter {

    private final BookCatalog catalog;           // required → constructor
    private @Nullable ReportFooter footer;       // optional → setter

    public ReportPrinter(BookCatalog catalog) {
        this.catalog = catalog;
    }

    @Autowired(required = false)                 // skipped when no ReportFooter
    ⇨ bean exists
    public void setFooter(ReportFooter footer) {
        this.footer = footer;
    }
}
```

Beklenen çıktı:

```
== 3.1 Constructor & setter injection
... BookService.findByAuthor took 139 µs
Joshua Bloch: [Effective Java, Java Puzzlers]
Kitap sayısı / Book count: 4
- Effective Java
- Java Puzzlers
```

- Modern Java in Action
- Spring in Action

Testi: book/BookServiceTest — Spring’e hiç ihtiyaç duymadan, new BookService(...) ile test edilir. Constructor injection’ın en büyük kazancı budur. book/ReportPrinterTest ise footer bean’i olan ve olmayan iki durumu dener.

3.2 @Component, @Bean ve @Configuration Proxy Modu

Amaç: Kendi sınıflarımızı @Component ile, başkalarının sınıflarını @Bean ile kaydetmek. @Configuration sınıflarının “full” ve “lite” modlarını ayırt etmek.

java.time.Clock JDK’ya ait bir sınıftır, üzerine @Component yazamayız. Bu yüzden onu bir @Bean metoduyla kaydederiz:

```
@Configuration(proxyBeanMethods = false)
public class AppConfig {

    @Bean
    Clock clock() {
        return Clock.systemDefaultZone(); // java.time.Clock: a JDK class, no
        ↪ @Component possible
    }
}
```

Varsayılan (**full**) modda Spring, @Configuration sınıfının bir CGLIB alt sınıfını (proxy) oluşturur. Bir @Bean metodunu başka bir @Bean metodu içinden çağırdığınızda proxy araya girer ve **var olan singleton’ı** döndürür:

```
@Configuration // proxyBeanMethods = true (default)
static class FullModeConfiguration {

    @Bean
    TaxRate fullModeTaxRate() {
        return new TaxRate(new BigDecimal("0.20"));
    }

    @Bean
    PriceCalculator fullModePriceCalculator() {
        // The CGLIB proxy intercepts this call and returns the existing singleton bean.
        return new PriceCalculator(fullModeTaxRate());
    }
}
```

proxyBeanMethods = false (**lite** mod) ile proxy oluşturulmaz. Başlangıç daha hızlıdır ve native image ile uyumludur. Ancak metod çağrısı artık sıradan bir Java çağrısıdır:

```
@Configuration(proxyBeanMethods = false)           // no proxy: faster startup, plain
↳ Java calls
static class LiteModeConfiguration {

    @Bean
    TaxRate liteModeTaxRate() {
        return new TaxRate(new BigDecimal("0.20"));
    }

    @Bean
    PriceCalculator liteModePriceCalculator() {
        // PITFALL: without a proxy this is a plain method call → a second, unmanaged
        ↳ TaxRate.
        return new PriceCalculator(liteModeTaxRate());
    }

    @Bean
    PriceCalculator liteModeSafePriceCalculator(TaxRate liteModeTaxRate) {
        // Correct lite-mode style: receive the bean as a method parameter.
        return new PriceCalculator(liteModeTaxRate);
    }
}
```

Beklenen çıktı:

```
== 3.2 @Configuration full vs lite mode
full mode shares the TaxRate bean: true
lite mode shares the TaxRate bean: false
```

Testi: configuration/ProxyModeTest — üç durumu da (full, lite tuzağı, lite doğru kullanım) ApplicationContextRunner ile doğrular.

★ İpucu

Spring Boot'un kendi auto-configuration sınıfları lite modu kullanır. Siz de @Bean metodları arasında bağımlılığı **metot parametresiyle** kurarsanız lite mod güvenlidir.

3.3 Bean Scope'ları

Amaç: Singleton ve prototype arasındaki farkı ve "singleton içinde prototype" tuzağını görmek.

ShoppingCart durum tutar (sepetteki ürünler). Her müşteriye ayrı bir sepet gerektiği için prototype scope kullanır. Container'dan her istendiğinde yeni bir nesne oluşturulur:

```
@Component
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class ShoppingCart {

    private final List<String> isbnns = new ArrayList<>(); // state → must not be shared
```

Tuzak: Prototype bir bean singleton bir bean'e enjekte edilirse, singleton bir kez olduğu için prototype da **yalnızca bir kez** oluşturulur. Bu durumda tüm müşteriler aynı sepeti paylaşır:

```
@Service
public class BrokenCheckoutService {

    private final ShoppingCart cart; // injected once, when this singleton is
    ↪ created

    public BrokenCheckoutService(ShoppingCart cart) {
        this.cart = cart;
    }

    public ShoppingCart startCheckout() {
        return cart; // every customer gets the SAME cart!
    }
}
```

Çözüm: Nesnenin kendisini değil, onu üreten bir ObjectProvider enjekte edin ve her ihtiyaçta yeni nesne isteyin:

```
@Service
public class CheckoutService {

    private final ObjectProvider<ShoppingCart> carts;

    public CheckoutService(ObjectProvider<ShoppingCart> carts) {
        this.carts = carts;
    }

    public ShoppingCart startCheckout() {
        return carts.getObject(); // a new cart on every call
    }
}
```

Beklenen çıktı:

== 3.3 Scopes

prototype in singleton → same cart twice: true

ObjectProvider → same cart twice: false

Testi: scope/CheckoutServiceTest

i Not

Web uygulamalarında request ve session scope'ları da vardır. Bunları Modül 03'te (Web MVC) göreceğiz.

3.4 Yaşam Döngüsü Callback'leri

Amaç: Bir bean hazır olduğunda çalışacak ve kapanırken temizlik yapacak kodu doğru yere koymak.

Constructor çalışırken bağımlılıklar hazırdır, ama “hazırlık” işini (ör. bir index kurmak) @PostConstruct metoduna koymak niyeti netleştirir. @PreDestroy ise context kapanırken çalışır:

```
@Component
public class TitleIndex {

    private final BookCatalog catalog;
    private final Map<String, String> titlesByLowerCase = new ConcurrentHashMap<>();

    public TitleIndex(BookCatalog catalog) {
        this.catalog = catalog; // 1. constructor: dependencies are injected
    }

    @PostConstruct
    void build() { // 2. after injection: safe to use dependencies
        catalog.findAll().stream()
            .map(Book::title)
            .forEach(title -> titlesByLowerCase.put(title.toLowerCase(Locale.ROOT),
                title));
    }

    @PreDestroy
    void clear() { // 3. on context close: release resources
        titlesByLowerCase.clear();
    }
}
```

Arka planda çalışan bir iş (ör. periyodik senkronizasyon) **tüm** bean'ler hazır olduktan sonra başlamalı ve kapanışta düzgünce durmalıdır. Bunun için SmartLifecycle kullanılır:

```
@Component
public class InventorySync implements SmartLifecycle {

    private static final Logger log = LoggerFactory.getLogger(InventorySync.class);

    private final Clock clock;
    private volatile boolean running;
    private volatile @Nullable Instant startedAt;

    public InventorySync(Clock clock) {
        this.clock = clock;
    }

    @Override
    public void start() { // called once the context has refreshed
        startedAt = clock.instant();
        running = true;
        log.info("Stok senkronizasyonu başladı / Inventory sync started");
    }

    @Override
    public void stop() { // called when the context closes
        running = false;
        log.info("Stok senkronizasyonu durdu / Inventory sync stopped");
    }

    @Override
    public boolean isRunning() {
        return running;
    }
}
```

Beklenen çıktı:

```
... InventorySync : Stok senkronizasyonu başladı / Inventory sync started
== 3.4 Lifecycle
TitleIndex.search("java") = [Effective Java, Java Puzzlers, Modern Java in Action]
InventorySync running: true
... InventorySync : Stok senkronizasyonu durdu / Inventory sync stopped
```

Testi: lifecycle/LifecycleTest — context kapandıktan sonra index'in temizlendiğini ve senkronizasyonun durduğunu doğrular.

3.5 Profiller ve Koşullu Bean'ler

Amaç: Aynı kodla, ortama veya konfigürasyona göre farklı bean'ler kullanmak.

@ConditionalOnProperty bir bean'i, bir property'nin değerine göre oluşturur. matchIfMissing = true bu seçeneği varsayılan yapar:

```
@Service
@ConditionalOnProperty(name = "bookstore.recommendations.strategy", havingValue =
    ↪ "popular", matchIfMissing = true)
public class PopularBooksRecommendation implements RecommendationService {
```

@Profile ile işaretli bir bean yalnızca o profil aktifken vardır. Örnek veriyi sadece geliştirme ortamında yüklemek için idealdir:

```
@Component
@Profile("dev") // run with: --spring.profiles.active=dev
public class DevSampleData implements ApplicationRunner {

    private final BookCatalog catalog;

    public DevSampleData(BookCatalog catalog) {
        this.catalog = catalog;
    }

    @Override
    public void run(@Nullable ApplicationArguments args) {
        catalog.save(new Book("978-605-000-001-1", "Kürk Mantolu Madonna", "Sabahattin
    ↪ Ali", new BigDecimal("45.00")));
        catalog.save(new Book("978-605-000-002-8", "Tutunamayanlar", "Öğuz Atay", new
    ↪ BigDecimal("110.00")));
    }
}
```

Çalıştırın: Varsayılan ayarlarla ve ardından dev profili ile budget stratejisini seçerek:

```
./mvnw -pl modules/01-core-container/lesson spring-boot:run \
-Dspring-boot.run.arguments="--spring.profiles.active=dev
    ↪ --bookstore.recommendations.strategy=budget"
```

Beklenen çıktı (ikinci çalıştırma):

```
== 3.5 Profiles & conditions
active profiles: [dev]
BudgetBooksRecommendation → [Kürk Mantolu Madonna, Java Puzzlers, Effective Java]
```

Testi: condition/RecommendationConditionTest, condition/DevSampleDataTest

3.6 BeanRegistrar ile Programatik Kayıt (Spring Framework 7)

Amaç: Hangi bean'lerin oluşacağına sıradan Java koduyla (döngü, if) karar vermek.

BeanRegistrar arayüzü, Spring Framework 7 ile geldi. Konfigürasyondaki kanal listesini okuyup her kanal için bir bean kaydeder. Eski BeanDefinitionRegistryPostProcessor yaklaşımından çok daha sadedir ve AOT/native image ile uyumludur:

```
public class NotificationChannelsRegistrar implements BeanRegistrar {

    private static final Map<String, Supplier<NotificationChannel>> KNOWN_CHANNELS =
        ↪ Map.of(
            "email", EmailChannel::new,
            "sms", SmsChannel::new,
            "push", PushChannel::new);

    @Override
    public void register(BeanRegistry registry, Environment env) {
        String[] configured = env.getProperty("bookstore.notifications.channels",
            ↪ String[].class, new String[] {"email"});
        List<String> names = Ar-
            ↪ rays.stream(configured).map(String::trim).filter(KNOWN_CHANNELS::containsKey).toList();

        for (String name : names) {
            Supplier<NotificationChannel> factory = KNOWN_CHANNELS.get(name);
            registry.registerBean(name + "Channel", NotificationChannel.class,
                spec -> spec.supplier(context -> factory.get()));
        }
    }
}
```

Registrar, bir @Configuration sınıfına @Import ile eklenir. List<NotificationChannel> enjeksiyonu, kaydedilen **tüm** kanalları toplar:

```
@Configuration(proxyBeanMethods = false)
@Import(NotificationChannelsRegistrar.class)
public class NotificationConfiguration {

    @Bean
    NotificationService notificationService(List<NotificationChannel> channels) {
        return new NotificationService(channels);
    }
}
```

Beklenen çıktı (application.yaml içinde channels: email,push):

```
== 3.6 BeanRegistrar
```

```
[[email] Kampanya başladı!, [push] Kampanya başladı!]
```

Testi: registrar/NotificationChannelsRegistrarTest

3.7 Uygulama Olayları (Events)

Amaç: Bir işin sonucunu, onu kimin dinlediğini bilmeden duyurmak (gevşek bağlılık).

BookService, kitap eklendikten sonra bir BookAddedEvent yayınlar:

```
public void add(Book book) {  
    catalog.save(book);  
    events.publishEvent(new BookAddedEvent(book)); // listeners run after this line  
}
```

Dinleyici, metodun parametre tipiyle hangi olayı dinleyeceğini belirtir. @Order birden fazla dinleyicinin sırasını belirler:

```
@Component  
public class NewBookAnnouncer {  
  
    private final NotificationLog log;  
  
    public NewBookAnnouncer(NotificationLog log) {  
        this.log = log;  
    }  
  
    @EventListener  
    @Order(1) // lower value → called earlier  
    public void announce(BookAddedEvent event) {  
        log.add("Yeni kitap / New book: " + event.book().title());  
    }  
}
```

condition özelliği bir SpEL ifadesidir. Yalnızca 100 ve üzeri fiyatlı kitaplar için çalışır:

```
@Component  
public class PremiumBookListener {  
  
    private final NotificationLog log;  
  
    public PremiumBookListener(NotificationLog log) {  
        this.log = log;  
    }  
}
```

```

    }

    @EventListener(condition = "#event.book().price() >= 100")
    @Order(2)
    public void onPremiumBook(BookAddedEvent event) {
        log.add("Premium kitap / Premium book: " + event.book().title());
    }
}

```

Beklenen çıktı:

```
== 3.7 Events
```

```
Yeni kitap / New book: Spring Boot: Up and Running
```

```
Premium kitap / Premium book: Spring Boot: Up and Running
```

Testi: event/BookEventListenersTest

! Önemli

@EventListener metotları varsayılan olarak **aynı thread’de ve hemen** çalışır. Olayın yalnızca veritabanı transaction’ı başarıyla commit edildikten sonra işlenmesi için @TransactionalEventListener kullanılır. Bunu Modül 05’te transaction’larla birlikte göreceğiz.

3.8 AOP ile Kesişen İlgiler

Amaç: Süre ölçümü gibi birçok sınıfı ilgilendiren kodu iş mantığından ayırmak.

@LogExecutionTime ile işaretlenen metotların süresini bir aspect ölçer. BookService.findByAuthor bu açıklamayı taşır:

```

@Aspect
@Component
public class ExecutionTimeAspect {

    private static final Logger log = LoggerFactory.getLogger(ExecutionTimeAspect.class);

    private final MethodTimings timings;

    public ExecutionTimeAspect(MethodTimings timings) {
        this.timings = timings;
    }

    @Around("@annotation(com.springbootedu.corecontainer.aop.LogExecutionTime)")
    public Object measure(ProceedingJoinPoint joinPoint) throws Throwable {

```

```

String method = joinPoint.getSignature().getDeclaringType().getSimpleName() + "."
    + joinPoint.getSignature().getName();
long start = System.nanoTime();
try {
    return joinPoint.proceed();    // call the real method
} finally {
    long micros = (System.nanoTime() - start) / 1_000;
    timings.record(method);
    log.info("{} took {} µs", method, micros);
}
}
}

```

Spring, BookService için bir **proxy** oluşturur. Diğer bean'ler gerçek nesne yerine bu proxy'yi alır ve çağrılar önce aspect'ten geçer.

Beklenen çıktı:

```

... ExecutionTimeAspect : BookService.findByAuthor took 139 µs
== 3.8 AOP
measured methods: [BookService.findByAuthor]

```

Testi: aop/ExecutionTimeAspectTest

! Uyarı

Spring Boot 4'te AOP starter'ının adı spring-boot-starter-aspectj'dir. Eski spring-boot-starter-aop artık yoktur.

4. Sık Yapılan Hatalar ve En İyi Pratikler

! Uyarı

Self-invocation: Bir sınıf kendi @LogExecutionTime metodunu this.findByAuthor(...) ile çağırırsa çağrı proxy'den geçmez ve aspect **çalışmaz**. Aynı durum @Transactional, @Async ve @Cacheable için de geçerlidir.

- **Yapın:** Zorunlu bağımlılıkları constructor ile alın ve alanları final yapın.
- **Yapmayın:** Field injection kullanmayın. Bağımlılıklar gizlenir ve sınıf Spring olmadan test edilemez.
- **Yapın:** Durum tutan nesneleri singleton yapmayın. Prototype gerekiyorsa ObjectProvider ile alın.
- **Yapmayın:** Lite modda (proxyBeanMethods = false) bir @Bean metodunu başka bir @Bean metodundan çağırmayın, parametre olarak isteyin.

- **Yapın:** Aynı arayüzün birden fazla uygulaması varsa seçimi `@ConditionalOnProperty` veya `@Profile` ile yapın, if blokları ile değil.
- **Yapmayın:** Constructor'lar arasında döngüsel bağımlılık ($A \rightarrow B \rightarrow A$) kurmayın. Spring Boot bunu varsayılan olarak hata sayar ve bu bir tasarım sorununa işaret eder.
- **Yapın:** Bir bean'in hangi koşulla oluştuğunu merak ettiğinizde uygulamayı `--debug` ile çalıştırın ve `conditions report`'a bakın.

5. Özet

- Spring container nesneleri (bean) oluşturur, birbirine bağlar ve yaşam döngülerini yönetir.
- Constructor injection varsayılan tercihtir. Setter injection yalnızca opsiyonel bağımlılıklar içindir.
- `@Component` kendi sınıflarınız, `@Bean` başkalarının sınıfları içindir. Lite modda bağımlılıkları metot parametresiyle alın.
- Singleton varsayılandır. Durum tutan bean'ler için `prototype` ve `ObjectProvider` kullanın.
- `@PostConstruct`/`@PreDestroy` ve `SmartLifecycle` başlangıç ve kapanış işlerini düzenler.
- `@Profile` ve `@ConditionalOnProperty` ortama göre bean seçer. `BeanRegistrar` karmaşık kayıt mantığı içindir.
- Olaylar bileşenleri birbirinden ayırır. AOP ise kesişen ilgileri iş kodundan ayırır.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Framework — The IoC Container](#)
- [Spring Framework — Programmatic Bean Registration](#)
- [Spring Framework — Bean Scopes](#)
- [Spring Framework — Application Events](#)
- [Spring Framework — Aspect Oriented Programming](#)
- [Spring Boot — Profiles](#)